

Experiment 10 – Implementation of Selenium Python and Behave BDD

Module 3 – Python BDD RESTful Automations

1. Module Name and Experiment Title

Module Name: Module 3 – Python BDD RESTful Automations

Experiment Number: 10

Experiment Title: Implementation of Selenium Python and Behave BDD

Programming Language: Python

Automation Tool: Selenium WebDriver

BDD Framework: Behave

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

2. Problem Statement

Develop an automated web testing framework using Selenium WebDriver and the Behave BDD framework. The automation should test the login functionality of the SauceDemo website using valid credentials.

The test scenario should be written in Gherkin syntax, while the corresponding step definitions should be implemented in Python. The automation must open the website, enter valid login credentials, click the login button, and verify that the user is redirected to the products page.

3. Objective

The main objectives of this experiment are:

1. To understand Behaviour-Driven Development (BDD) using the Behave framework.
 2. To integrate Selenium WebDriver with Behave using Python.
 3. To write test scenarios using Gherkin syntax.
 4. To implement step definitions for each test step.
 5. To automate the login functionality of the SauceDemo website.
 6. To verify successful login using assertions.
 7. To capture screenshots at important stages of test execution.
-

4. Tools, Software and Concepts Used

Tools and Software

- Python
- Selenium WebDriver
- Behave BDD Framework
- Google Chrome
- Visual Studio Code
- Windows PowerShell

Concepts Used

- Behaviour-Driven Development (BDD)
- Gherkin syntax
- Feature files
- Scenarios and steps
- Step definitions
- Selenium WebDriver
- Explicit waits using `WebDriverWait`
- Assertions
- Automated screenshot capture

5. Source Code / Implementation Steps

Step 1: Project Structure

The project was organized into the following directory structure:

Assignment 1 - Selenium Python and Behave BDD/

```
├── features/
│   ├── login.feature
│   ├── environment.py
│   └── steps/
│       └── login_steps.py
├── screenshots/
│   ├── 01_login_page.png
│   ├── 02_credentials_entered.png
│   └── 03_products_page.png
└── requirements.txt
```

Step 2: Install Required Libraries

The following dependencies were included in the `requirements.txt` file:

```
selenium
behave
```

Install the required libraries using the following command:

```
pip install -r requirements.txt
```

Step 3: Create the Feature File

File: features/login.feature

The feature file describes the login functionality using Gherkin syntax.

```
Feature: SauceDemo Login
```

```
    Scenario: Successful login with valid credentials
        Given the user opens the SauceDemo website
        When the user enters valid username and password
        And the user clicks the login button
        Then the user should be redirected to the products page
```

Step 4: Configure the WebDriver

File: features/environment.py

The environment file initializes the Chrome browser before each scenario and closes it after execution.

```
from selenium import webdriver

def before_scenario(context, scenario):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()

def after_scenario(context, scenario):
    context.driver.quit()
```

Step 5: Implement the Step Definitions

File: features/steps/login_steps.py

The step definitions connect the Gherkin steps to Selenium WebDriver actions.

```
import os
import time

from behave import given, when, then
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

os.makedirs("screenshots", exist_ok=True)

@given("the user opens the SauceDemo website")
def open_website(context):
    context.driver.get("https://www.saucedemo.com/")
    time.sleep(2)
```

```

context.driver.save_screenshot(
    "screenshots/01_login_page.png"
)

print("SauceDemo website opened successfully.")

@when("the user enters valid username and password")
def enter_credentials(context):
    wait = WebDriverWait(context.driver, 15)

    username = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "user-name")
        )
    )

    username.send_keys("standard_user")
    time.sleep(2)

    password = context.driver.find_element(
        By.ID, "password"
    )

    password.send_keys("secret_sauce")
    time.sleep(2)

    context.driver.save_screenshot(
        "screenshots/02_credentials_entered.png"
    )

    print("Login credentials entered successfully.")

@when("the user clicks the login button")
def click_login(context):
    context.driver.find_element(
        By.ID, "login-button"
    ).click()

    time.sleep(2)

    print("Login button clicked.")

@then("the user should be redirected to the products page")
def verify_login(context):
    wait = WebDriverWait(context.driver, 15)

    wait.until(
        EC.visibility_of_element_located(
            (By.CLASS_NAME, "inventory_list")
        )
    )

    assert "/inventory.html" in context.driver.current_url

    context.driver.save_screenshot(
        "screenshots/03_products_page.png"
    )

```

```
print("Login successful!")
print("Products page verified.")
```

Step 6: Execute the Test

Open PowerShell in the project directory and execute the following command:

```
behave --no-capture
```

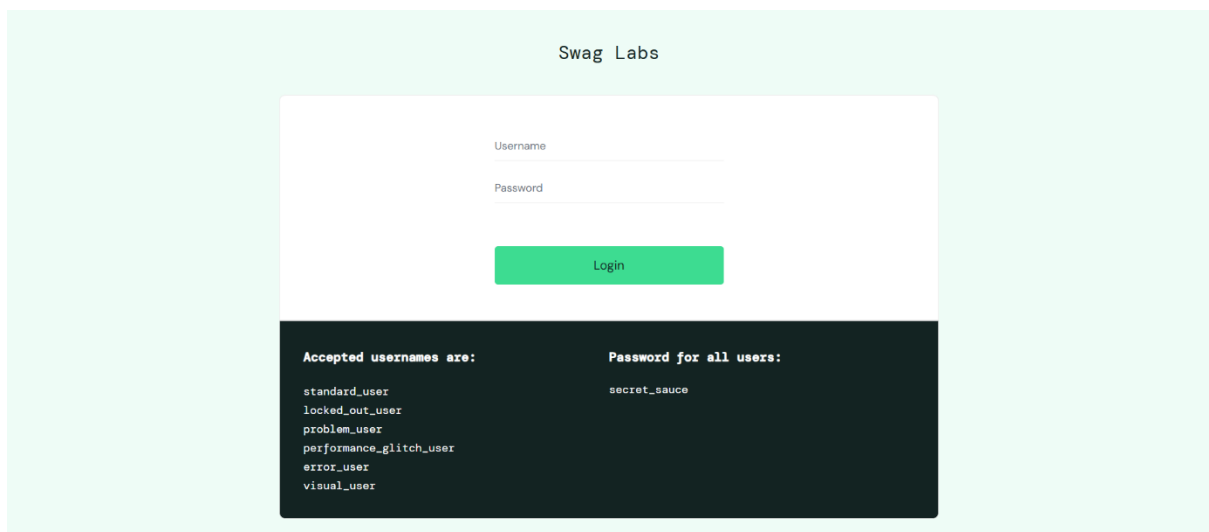
The `--no-capture` option displays the print statements generated during test execution in the terminal.

6. Output with Clearly Labelled Screenshots

The automation was executed successfully on the SauceDemo website. The following screenshots show the important stages of execution.

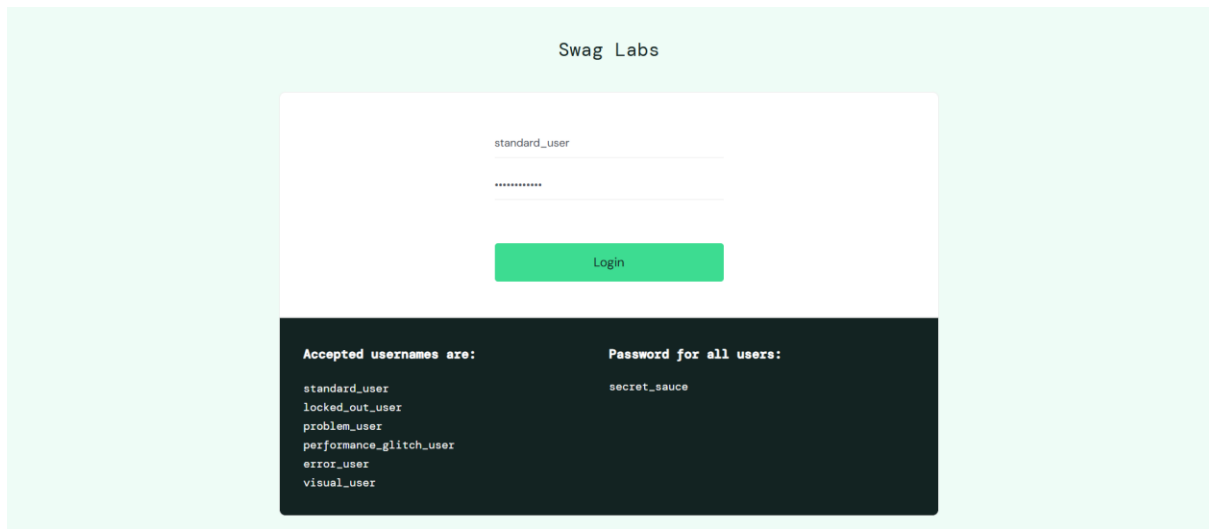
Screenshot 1: SauceDemo Login Page

Description: The SauceDemo login page was opened successfully by Selenium WebDriver. The login form and accepted test credentials were visible.



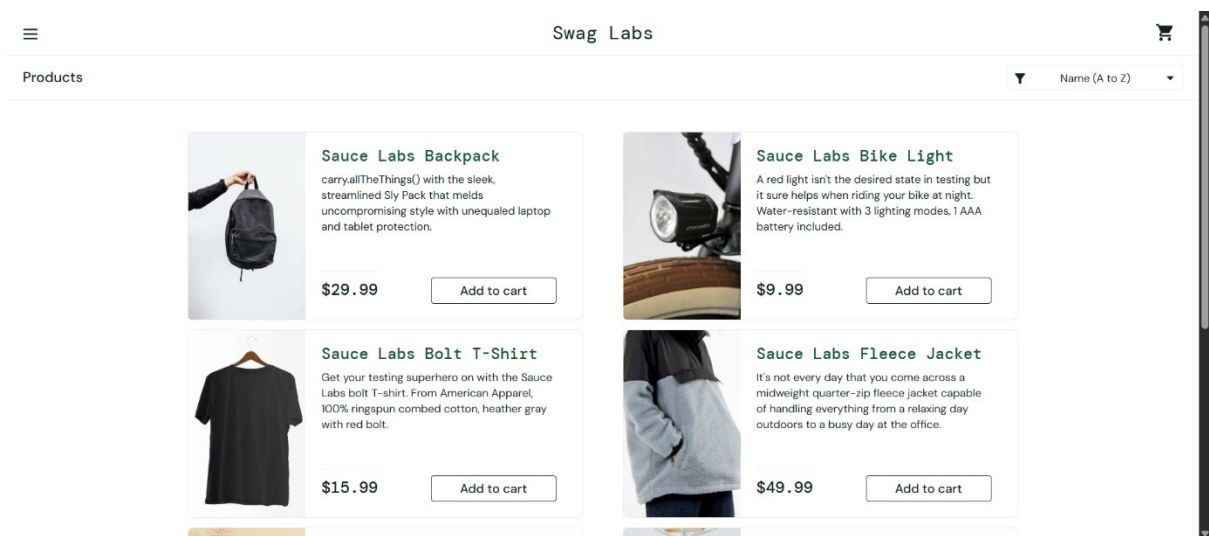
Screenshot 2: Login Credentials Entered

Description: The automation entered the valid username `standard_user` and password `secret_sauce` into the login form before submitting it.



Screenshot 3: Products Page After Successful Login

Description: After clicking the login button, the automation successfully navigated to the products page. The presence of the inventory list and the `/inventory.html` URL confirmed successful login.



Execution Output

The following output was displayed in PowerShell after running the Behave test:

```
USING RUNNER: behave.runner:Runner
```

```
Feature: SauceDemo Login
```

```
Scenario: Successful login with valid credentials
```

```
Given the user opens the SauceDemo website
```

```
When the user enters valid username and password
```

```
And the user clicks the login button
```

```
Then the user should be redirected to the products page
```

```
Products page verified.
```

```
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
```

Took 0min 9.193s

Execution Summary

Metric	Result
Features Passed	1
Scenarios Passed	1
Steps Passed	4
Failed	0
Skipped	0
Execution Time	9.193 seconds

7. Result, Observation and Conclusion

Result

The Selenium Python and Behave BDD automation was successfully implemented and executed on the SauceDemo website. The test scenario completed successfully, with all four steps passing and no failures or skipped steps.

Observation

1. The Behave framework successfully executed the test scenario written in Gherkin syntax.
2. Selenium WebDriver opened the SauceDemo website and entered the valid login credentials.
3. The login button was clicked successfully.
4. The automation verified the products page using an explicit wait and URL assertion.
5. Screenshots were captured at three important stages of execution.
6. The Chrome browser was closed automatically after the scenario finished.

Conclusion

This experiment demonstrated how Selenium WebDriver can be integrated with the Behave BDD framework using Python. The login scenario was defined in a Gherkin feature file, and its steps were implemented using Python step definitions.

The successful execution of the test confirms that the implemented automation can verify the SauceDemo login functionality. The experiment also provided practical experience with BDD concepts, browser automation, assertions, explicit waits, and screenshot capture.